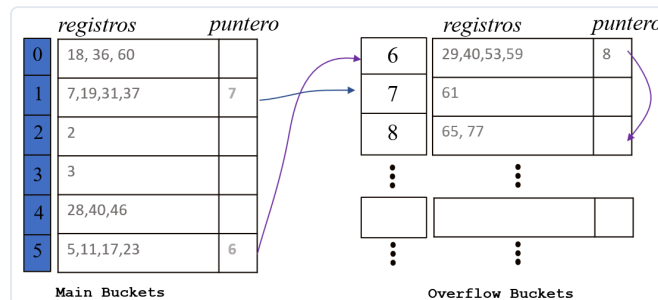


Laboratorio 05: Hash File

P1 (6 pts). Static Hashing (Implementación)

Implementar Static Hashing con buckets y desbordamiento encadenado.

1. Diseñar la estructura del bucket y la clase que gestiona el archivo.
2. Inserción: Si el bucket está lleno, crear y enlazar un bucket overflow.
3. Búsqueda: Localizar el bucket y buscar el registro, incluyendo buckets overflow.
4. Eliminación: Eliminar el registro. ¿Qué hacer si un bucket queda vacío?



P2 (6 pts). Extendible Hashing (Ejercicio manual)

Aplicar el algoritmo de Extendible Hashing al siguiente conjunto de claves:

keys: 10, 13, 34, 6, 23, 12, 15, 73, 28, 19, 67, 17, 41, 87, 57, 27, 11

a) (2 pts) Proceso de inserción de datos

- Para calcular la posición del bucket, utilice la función:

$$pos_bucket = Binary(hash(key) \% 2^D)$$
- Factor de bloque: **FB = 3**.
- La profundidad global inicia en $D = 2$ y se incrementa cada vez que se realiza un rehashing ($D = 3, D = 4, \dots$).
- El índice hash debe direccionar a los buckets correspondientes en el archivo de datos. El archivo inicia con dos buckets (0 y 1) y se incrementa el número de buckets cuando ocurre un overflow.
- Cuando un bucket se desborda, aplique Split y actualice el índice con la nueva entrada referenciando al bucket creado. Si se alcanza la profundidad global, encadene los buckets desbordados.
- Mantenga un encadenamiento máximo de $K = 1$; es decir, solo se permite un bucket encadenado. Si se supera, aplique rehashing.

Resolver gráficamente paso a paso.

b) (2 pts) Algoritmo de búsqueda

- Proponga el algoritmo de alto nivel necesario para la búsqueda de un registro, considerando todos los casos posibles.

c) (2 pts) Algoritmo de eliminación

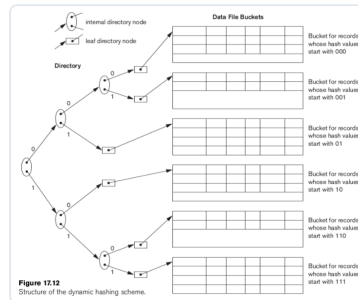
- Proponga el algoritmo de alto nivel para eliminar un registro, considerando todos los casos. ¿Cómo gestionar los buckets libres?

P3 (6 pts). Trie Hashing (Ejercicio manual)

Resolver manualmente el caso anterior, pero usando un Trie (árbol digital) para mantener el directorio como un árbol binario, donde los nodos hoja guardan los punteros a los buckets en el archivo de datos.

Trie Hashing, Litwin, W. (1981), ACM SIGMOD

- Considerar como profundidad global $D = 3$.
- El algoritmo inicia con dos buckets (0 y 1).
- Para ubicar un registro en su bucket correspondiente, tome los d -primeros dígitos del hash (d es el nivel del árbol).
- Cuando un bucket se desborda, cree dos nodos hijos incrementando el nivel de bits (por ejemplo, 01 $\rightarrow$ 010, 011).



1. Dibuje el árbol final.
2. Diseñe el algoritmo de inserción.
3. Diseñe el algoritmo de búsqueda.
4. (Opcional) Diseñe el algoritmo de eliminación.

P4 (2 pts). Linear Hashing (Ejercicio manual)

Investigue cómo aplicar la técnica de Linear Hashing a nivel de buckets:

Linear Hashing: a new tool for file and table addressing. Litwin, W. (1980)

1. Elabore paso a paso el Linear Hashing usando los datos de la pregunta P2.
2. Diseñe el algoritmo de inserción.
3. Diseñe el algoritmo de búsqueda.

Entregable

- **Informe** que incluya la solución estructurada de las cuatro preguntas y el análisis de ventajas y desventajas de cada técnica.